

Проект анализатор треков

Исполнители: Мельников Л. А., Копылов М. А., Беляков А. К.

Цель проекта: Разработка Telegram бота, способного анализировать аудиофайлы, извлекать их акустические характеристики и предоставлять рекомендации похожей музыки на основе полученных метрик.

Мельников Л. А. отвечает за микросервис с загрузкой файлов в telegram бота и серверную часть.

Копылов М. А. отвечает за создание микросервиса анализатора аудиофайлов и извлечение метрик(характеристик) из них.

Беляков А. К. отвечает за микросервис подбора рекомендаций по метрикам.

Планируемые задачи:

- POST /api/upload принимает файл и сохраняет в MinIO
- GET /health возвращает статус 200
- Валидация типов файлов работает
- JWT токены проверяются
- Unit - тестирование
- Документация

Технологический стек: Rust, Python, Postgresql

<https://habr.com/ru/companies/otus/articles/757130/>

Лабораторная работа №2: проектирование архитектуры, разработка telegram бота

1.1. Анализ требований и функциональная спецификация

- Определение основных функций бота:
 - Прием аудиофайлов (mp3, wav, ogg, m4a)
 - Анализ аудиохарактеристик (темп, энергия, танцевальность и др.)
 - Сохранение истории анализов
 - Поиск похожих треков
 - Формирование рекомендаций
- Составление диаграммы вариантов использования
- Определение нефункциональных требований (производительность, безопасность)

1.2. Проектирование базы данных

- Разработка ER-диаграммы
- Создание схемы данных:
 - Таблица пользователей (users)
 - Таблица аудиофайлов (audio_files)
 - Таблица аудиохарактеристик (audio_features)
 - Таблица рекомендаций (recommendations)
 - Таблица очередей обработки (processing_queue)
- Определение индексов и связей между таблицами

1.3. Проектирование микросервисной архитектуры

- Разработка схемы взаимодействия сервисов
- Определение API-интерфейсов между компонентами
- Проектирование системы очередей для асинхронной обработки
- Создание диаграммы развертывания

1.4. Настройка окружения Rust

- Установка Rust и Cargo
- Настройка зависимостей (Cargo.toml)

1.5. Разработка базовой структуры бота

- Создание модульной архитектуры:
 - main.rs — точка входа
 - handlers/ — обработчики команд
 - services/ — бизнес-логика
 - models/ — модели данных
 - utils/ — вспомогательные функции
 - config/ — конфигурация

1.6. Реализация команд бота

- Команда /start — регистрация пользователя, приветствие
- Команда /help — справка по использованию
- Команда /analyze — анализ аудиофайла
- Команда /recommend — получение рекомендаций
- Команда /history — история анализов

- Команда /stats — статистика использования

1.7. Обработка медиафайлов

- Прием и валидация аудиофайлов
- Загрузка ZIP-архивов с множеством файлов
- Webhook-уведомления для клиентов
- Обнаружение дубликатов по хешу файла
- Проверка заголовков MP3/FLAC/WAV
- Прогресс-бар для обработки
- Сохранение файлов во временное хранилище
- Отправка файлов в очередь на обработку
- Отображение прогресса обработки

1.8. Интеграция с PostgreSQL

- Создание пула соединений
- Реализация CRUD-операций
- Миграции базы данных

Лабораторная работа №3: разработка микросервиса анализа аудио и построение системы рекомендаций

3.1. Настройка Python-окружения

- Создание виртуального окружения
- Установка зависимостей:

3.2. Разработка модуля извлечения признаков

- Реализация функций для извлечения:
 - Темп (BPM) — определение скорости музыки
 - Энергия — интенсивность звука
 - Танцевальность — пригодность для танцев
 - Валентность — эмоциональная окраска (позитив/негатив)

- Акустичность — степень "живого" звучания
- Инструментальность — наличие вокала
- Разговорность — содержание речи
- Громкость — уровень звука
- Тональность — музыкальный ключ
- Лад — мажор/минор

3.3. Разработка алгоритмов анализа

- Предобработка аудиосигнала
- Извлечение MFCC (мел-частотные кепстральные коэффициенты)
- Спектральный анализ
- Ритмический анализ
- Гармонический анализ

3.4. Создание REST API

- Эндпоинт для получения статуса обработки
- Эндпоинт для получения результатов анализа
- Эндпоинт для проверки работоспособности

3.5. Разработка модуля нормализации данных

- Масштабирование признаков
- Обработка выбросов
- Заполнение пропущенных значений

3.6. Реализация алгоритмов схожести

- Косинусная схожесть — основной метод
- Евклидово расстояние — альтернативный метод
- Корреляция Пирсона — для сравнения паттернов

3.7. Разработка системы взвешивания признаков

- Определение весов для каждой характеристики
- Настройка весов на основе экспертных оценок
- Возможность калибровки весов

3.8. Создание модели рекомендаций

- Построение матрицы признаков
- Кластеризация треков
- Индексация для быстрого поиска

3.9. Реализация API для рекомендаций

- Эндпоинт получения рекомендаций по ID трека
- Эндпоинт получения похожих треков по характеристикам
- Эндпоинт для пакетных рекомендаций

3.10. Кэширование результатов

- Интеграция с Redis
- Настройка времени жизни кэша
- Инвалидация кэша при обновлении данных

Лабораторная работа №4: тестирование, оптимизация и документация

4.1. Модульное тестирование

- Rust: тесты для каждого модуля бота
- Python: pytest для микросервисов
- Тестирование граничных случаев

4.2. Интеграционное тестирование

- Тестирование взаимодействия между сервисами
- Проверка работы очередей сообщений
- Тестирование базы данных

4.3. Нагрузочное тестирование

- Проверка производительности при параллельных запросах
- Оптимизация узких мест

- Настройка пулов соединений

4.4. Тестирование качества рекомендаций

- Сбор тестовой выборки
- Оценка точности рекомендаций
- A/B тестирование различных алгоритмов

4.5. Оптимизация кода

- Рефакторинг
- Оптимизация запросов к БД
- Улучшение алгоритмов анализа

4.6. Техническая документация

- Документирование API
- Описание архитектуры
- Инструкция по развертыванию
- Документация по коду

4.7. Контейнеризация

- Создание Dockerfile для каждого сервиса
- Настройка docker-compose для локальной разработки
- Настройка сетей и томов